

IF2211 STRATEGI ALGORITMA
SEMESTER II TAHUN 2025/2026

Laporan Tugas Besar 2

*Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme
Penelusuran CSS pada Pohon Document Object Model*



Disusun oleh Kelompok: **BOELANGOSONK-V2**

13524001 Samuelson Dharmawan Tanuraharja
13524036 Edward David Rumahorbo
13524056 Reinhard Alfonzo Hutabarat

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

1	Deskripsi Tugas	3
1.1	Latar Belakang	3
1.2	Tujuan	3
1.3	Ruang Lingkup	4
2	Landasan Teori	5
2.1	Document Object Model (DOM)	5
2.2	HTML Parsing	5
2.3	CSS Selector	6
2.3.1	Selektor Dasar	6
2.3.2	Kombinator	6
2.4	Breadth-First Search (BFS)	7
2.4.1	Mekanisme Kerja	7
2.4.2	Sifat dan Kompleksitas	8
2.5	Depth-First Search (DFS)	8
2.5.1	Mekanisme Kerja	8
2.5.2	Sifat dan Kompleksitas	9
2.5.3	Perbandingan BFS dan DFS	9
2.6	Lowest Common Ancestor dengan Binary Lifting	10
2.6.1	Definisi Formal	10
2.6.2	Teknik Binary Lifting	10
2.6.3	Kompleksitas dan Konteks Penerapan	11
3	Aplikasi BFS dan DFS	12
3.1	Alur Kerja BFS pada Penelusuran DOM	12
3.2	Alur Kerja DFS pada Penelusuran DOM	13
3.3	Mekanisme Pencocokan CSS Selector	14
3.4	Paralelisasi Pencocokan dengan Goroutine	15
3.5	Animasi Traversal	17
3.5.1	Alur Umum	17
3.5.2	State Management pada useTraversal	17
3.5.3	Komputasi animatedData	17
3.5.4	Komponen PlaybackControls	18
3.6	Penerapan LCA Binary Lifting pada DOM	18
3.6.1	Pembangunan Tabel (Build)	18
3.6.2	Penyimpanan pada Store	18
3.6.3	Query LCA via API	18
4	Implementasi dan Pengujian	19
4.1	Lingkungan Pengembangan	19
4.2	Struktur Program	19
4.3	Implementasi Backend	21
4.3.1	Arsitektur dan Integrasi Komponen	21
4.3.2	Alur Pemrosesan Query	21
4.3.3	Integrasi Frontend-Backend	22
4.4	Implementasi Frontend	22
4.4.1	Arsitektur dan Integrasi Komponen	22
4.4.2	Alur Pemrosesan dan Visualisasi	23
4.4.3	Integrasi Komponen Pendukung	23
4.5	Pengujian	23
5	Kesimpulan dan Saran	32
5.1	Kesimpulan	32
5.2	Saran	32
	Pernyataan Tidak Melakukan Kecurangan	33



Daftar Pustaka	34
Lampiran	35

1 Deskripsi Tugas

1.1 Latar Belakang

Perkembangan internet yang pesat telah menjadikan dokumen HTML (*HyperText Markup Language*) sebagai medium utama penyampaian informasi di *World Wide Web*. Setiap halaman web yang diakses melalui peramban (*browser*) dibangun dari sebuah dokumen HTML yang terstruktur secara hierarkis. Struktur hierarkis ini direpresentasikan oleh peramban sebagai *Document Object Model* (DOM), yakni sebuah pohon yang setiap simpulnya (*node*) berkorespondensi dengan elemen, atribut, atau konten teks pada dokumen HTML [2].

Manajemen tampilan setiap halaman web dilakukan melalui *Cascading Style Sheets* (CSS), yang memanfaatkan mekanisme *CSS Selector* untuk mengidentifikasi elemen-elemen tertentu pada pohon DOM dan menerapkan aturan gaya (*style rules*) kepada elemen-elemen tersebut [4]. Proses pencarian elemen berdasarkan *CSS Selector* pada dasarnya merupakan masalah penelusuran pohon (*tree traversal*), di mana setiap simpul perlu diperiksa untuk menentukan apakah sesuai dengan kriteria selektor yang diberikan.

Dalam lingkungan peramban, penelusuran ini biasanya diselesaikan secara implisit melalui API bawaan seperti `document.querySelectorAll()`. Namun, pemahaman mendalam tentang cara kerja algoritma di balik proses ini memiliki nilai edukatif dan aplikatif yang signifikan, antara lain dalam pembuatan *web scraper*, alat pengujian antarmuka (*UI testing tools*), serta sistem analisis struktur halaman web secara otomatis.

Dua algoritma penelusuran graf yang fundamental, yaitu *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS), merupakan landasan teoritis yang tepat untuk menyelesaikan masalah penelusuran DOM [1]. BFS menelusuri pohon secara berlapis-lapis (*level-order*), sedangkan DFS menelusuri sedalam mungkin sebelum melakukan *backtracking*. Karakteristik keduanya yang berbeda memberikan fleksibilitas dalam memilih strategi penelusuran yang paling sesuai dengan kebutuhan dan karakteristik pohon DOM yang dihadapi.

Berdasarkan latar belakang tersebut, Tugas Besar 2 IF2211 Strategi Algoritma ini bertujuan mengembangkan sebuah aplikasi berbasis web yang mengimplementasikan algoritma BFS dan DFS untuk menelusuri dan mencari elemen pada struktur pohon DOM berdasarkan *CSS Selector* yang diberikan oleh pengguna. Aplikasi ini dilengkapi pula dengan fitur visualisasi pohon DOM, animasi penelusuran langkah per langkah (*step-by-step*), serta fitur *Lowest Common Ancestor* (LCA) menggunakan teknik *Binary Lifting* sebagai komponen tambahan yang memperkaya fungsionalitas sistem.

1.2 Tujuan

Tujuan pengembangan aplikasi pada Tugas Besar 2 ini adalah sebagai berikut.

1. Mengembangkan aplikasi berbasis web yang terdiri dari *frontend* menggunakan React dengan TypeScript serta *backend* menggunakan bahasa pemrograman Go dengan kerangka kerja Gin.
2. Mengimplementasikan fitur pengambilan HTML (*scraping*) dari URL yang disediakan pengguna maupun dari masukan teks HTML secara langsung.
3. Membangun *parser* HTML yang mengubah dokumen HTML menjadi struktur data pohon DOM *custom* dalam memori, dengan memanfaatkan paket `golang.org/x/net/html` sebagai lapisan *parsing* tingkat rendah dan mengimplementasikan sendiri logika konstruksi struktur *DOMNode* yang digunakan oleh sistem.
4. Mengimplementasikan algoritma BFS dan DFS untuk menelusuri pohon DOM dan menemukan elemen-elemen yang sesuai dengan *CSS Selector* yang diberikan, dengan pilihan untuk menampilkan n hasil pertama atau seluruh hasil yang ditemukan.
5. Menampilkan visualisasi interaktif dari struktur pohon DOM beserta informasi kedalaman maksimumnya.
6. Menampilkan sorotan (*highlight*) pada jalur penelusuran yang ditempuh oleh algoritma yang dipilih.

7. Menampilkan metrik kinerja berupa waktu pencarian dan jumlah simpul yang dikunjungi selama proses penelusuran berlangsung.
8. Menyimpan catatan penelusuran (*traversal log*) yang memuat informasi tahapan penelusuran yang telah dilakukan.
9. (*Bonus*) Menampilkan animasi penelusuran pohon DOM secara langkah per langkah melalui pemutaran ulang log penelusuran (*in-memory log replay*) pada sisi klien.
10. (*Bonus*) Mengimplementasikan algoritma LCA dengan teknik *Binary Lifting* untuk menemukan simpul leluhur terdekat yang sama dari dua simpul yang dipilih pada pohon DOM.
11. (*Bonus*) Mengimplementasikan *multithreading* menggunakan goroutine untuk memparalelkan proses pencocokan (*matching*) *CSS Selector* pada setiap simpul yang dikunjungi.
12. (*Bonus*) Mengemas aplikasi menggunakan Docker untuk memudahkan proses *deployment* yang konsisten.
13. (*Bonus*) Melakukan *deployment* aplikasi pada Microsoft Azure Virtual Machine sehingga dapat diakses secara publik melalui internet.

1.3 Ruang Lingkup

Ruang lingkup pengembangan aplikasi pada tugas besar ini mencakup hal-hal berikut.

Masukan (*Input*):

- URL halaman web yang akan di-*scraping*, atau teks HTML mentah yang dimasukkan langsung oleh pengguna.
- Pilihan algoritma penelusuran: BFS atau DFS.
- *CSS Selector* yang ingin dicari pada pohon DOM.
- Jumlah hasil yang ingin ditampilkan: n hasil pertama atau seluruh hasil yang ditemukan.

Keluaran (*Output*):

- Visualisasi interaktif dari pohon DOM beserta informasi kedalaman maksimum pohon.
- Sorotan pada elemen-elemen yang ditemukan dan jalur penelusuran yang telah dilalui.
- Metrik kinerja: waktu pencarian (dalam milidetik) dan jumlah simpul yang dikunjungi.
- Berkas *traversal log* yang dapat diunduh.

CSS Selector yang Didukung:

- Selektor dasar: selektor *tag*, selektor kelas (*class*, diawali `.`), selektor ID (diawali `#`), dan selektor universal (`*`).
- Kombinator: *child* (`>`), *descendant* (spasi), *adjacent sibling* (`+`), dan *general sibling* (`~`).

Tumpukan Teknologi (*Tech Stack*):

- *Backend*: Go 1.25 dengan Gin Web Framework, dikontainerisasi menggunakan Docker.
- *Frontend*: React 19 dengan TypeScript, dibangun menggunakan Vite dan Bun sebagai *run-time*.
- Visualisasi pohon: pustaka `@xyflow/react`.
- Animasi penelusuran langkah per langkah: *in-memory log replay* pada sisi klien.

2 Landasan Teori

2.1 Document Object Model (DOM)

Menurut *DOM Living Standard* yang dikelola oleh WHATWG, *Document Object Model* (DOM) adalah antarmuka pemrograman (*API*) yang merepresentasikan dokumen HTML atau XML sebagai struktur pohon, di mana setiap komponen dokumen dimodelkan sebagai sebuah objek yang dapat diakses dan dimanipulasi oleh program [2]. Dalam konteks dokumen HTML, setiap tag HTML direpresentasikan sebagai sebuah *node* pada pohon DOM.

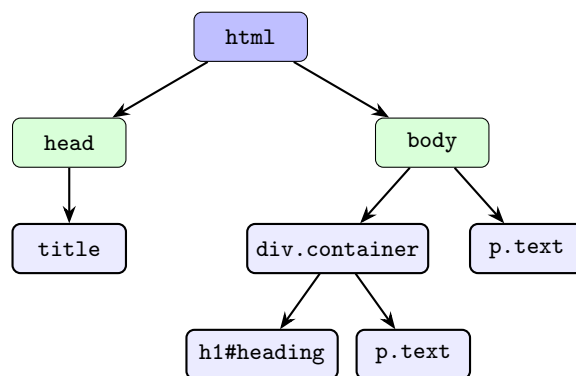
Jenis-jenis *node* yang umum dijumpai dalam pohon DOM meliputi:

- **Element Node**: merepresentasikan tag HTML seperti `<div>`, `<p>`, dan `<h1>`. Jenis *node* ini menjadi fokus utama dalam penelusuran berbasis *CSS Selector*.
- **Text Node**: merepresentasikan konten teks yang berada di dalam sebuah elemen.
- **Comment Node**: merepresentasikan komentar HTML yang ditulis di antara tag `<!--` dan `-->`.

Hubungan antar *node* pada pohon DOM mencerminkan hubungan hierarki elemen pada dokumen HTML:

- **Parent-Child**: *node* yang berada langsung satu tingkat di atas disebut *parent*, dan *node* yang berada langsung satu tingkat di bawah disebut *child*.
- **Sibling**: *node-node* yang memiliki *parent* yang sama disebut *sibling* (saudara).
- **Ancestor-Descendant**: generalisasi dari hubungan *parent-child* untuk lebih dari satu tingkatan hierarki.

Setiap *element node* memiliki atribut-atribut yang relevan untuk pencocokan *CSS Selector*, yaitu nama tag (*tag name*), daftar kelas (*class list*), dan identitas unik (*id*). Gambar 1 mengilustrasikan struktur pohon DOM dari sebuah dokumen HTML sederhana.



Gambar 1: Contoh struktur pohon DOM dari sebuah dokumen HTML sederhana

Pada Gambar 1, `html` adalah *root* dari pohon. Elemen `head` dan `body` adalah *child* dari `html` dan merupakan *sibling* satu sama lain. Elemen `p.text` di dalam `div.container` merupakan *sibling* dari `h1#heading`, dan keduanya merupakan *descendant* dari `body`.

2.2 HTML Parsing

HTML Parsing adalah proses mengonversi teks mentah dokumen HTML menjadi struktur data pohon DOM yang dapat diproses oleh program. Standar WHATWG HTML Living Standard [3] mendefinisikan proses ini secara detail sebagai mesin keadaan (*state machine*) yang terdiri dari dua tahap utama:

1. Tokenisasi (*Tokenization*):

Pada tahap ini, aliran karakter HTML diproses secara berurutan untuk menghasilkan token-token diskrit. Jenis token yang dihasilkan meliputi:

- **Start Tag Token:** menandai awal elemen HTML, beserta atribut-atributnya (misalnya `<div class="box">`).
- **End Tag Token:** menandai akhir elemen HTML (misalnya `</div>`).
- **Character Token:** merepresentasikan konten teks di antara tag.
- **Comment Token** dan **DOCTYPE Token:** komentar dan deklarasi tipe dokumen.

2. Konstruksi Pohon (*Tree Construction*):

Token-token yang dihasilkan kemudian diproses secara berurutan untuk membangun pohon DOM menggunakan sebuah tumpukan (*stack*) terbuka. Prinsip kerjanya adalah sebagai berikut:

- Saat menerima *start tag token*, buat *node* baru dan jadikan sebagai *child* dari *node* yang berada di puncak *stack*, lalu *push node* baru tersebut ke dalam *stack*.
- Saat menerima *end tag token*, *pop node* dari puncak *stack* (elemen dinyatakan tertutup).
- Saat menerima *character token*, tambahkan konten teks sebagai *text node* pada *node* yang berada di puncak *stack*.

Pada implementasi tugas besar ini, kedua tahap di atas dilakukan secara mandiri pada modul `parser.go`. Tahap tokenisasi memanfaatkan `html.NewTokenizer` dari paket `golang.org/x/net/html` untuk menghasilkan aliran token dari teks HTML mentah. Tahap konstruksi pohon kemudian dilakukan sepenuhnya oleh kode sistem: setiap token diproses satu per satu menggunakan tumpukan (*stack*) eksplisit yang dikelola sendiri, membangun hierarki `DOMNode custom` yang memuat atribut-atribut yang diperlukan oleh mesin penelusuran dan visualisasi.

2.3 CSS Selector

CSS Selector adalah pola yang digunakan dalam CSS untuk menentukan elemen-elemen mana pada pohon DOM yang akan dikenai aturan gaya. Spesifikasi W3C Selectors Level 4 [4] mendefinisikan berbagai jenis selektor yang tersedia. Pada implementasi tugas besar ini, jenis-jenis selektor berikut didukung:

2.3.1 Selektor Dasar

Type Selector (Selektor Tag)

Memilih semua elemen dengan nama tag tertentu. Contoh: `p` memilih semua elemen `<p>`; `div` memilih semua elemen `<div>`.

Class Selector (Selektor Kelas)

Memilih semua elemen yang memiliki kelas tertentu pada atribut `class`-nya, ditulis dengan awalan titik (`.`). Contoh: `.box` memilih setiap elemen yang atribut `class`-nya mengandung kata "box".

ID Selector (Selektor ID)

Memilih elemen yang memiliki ID tertentu, ditulis dengan awalan tanda pagar (`#`). Contoh: `#header` memilih elemen dengan `id="header"`.

Universal Selector (Selektor Universal)

Memilih semua elemen tanpa terkecuali, ditulis dengan tanda asterisk (`*`).

2.3.2 Kombinator

Kombinator menggabungkan dua buah selektor untuk mengekspresikan hubungan struktural tertentu antara dua elemen pada pohon DOM.

Descendant Combinator (spasi)

Pola `A B` memilih semua elemen `B` yang merupakan *descendant* dari elemen `A`, tidak harus merupakan *child* langsung.

Child Combinator ($>$)

Pola $A > B$ memilih semua elemen B yang merupakan *child* langsung dari elemen A .

Adjacent Sibling Combinator ($+$)

Pola $A + B$ memilih elemen B yang berada tepat setelah elemen A dan memiliki *parent* yang sama.

General Sibling Combinator ($\sim$)

Pola $A \sim B$ memilih semua elemen B yang berada setelah elemen A (tidak harus tepat sesudahnya) dan memiliki *parent* yang sama.

Tabel 1: Ringkasan CSS Selector yang didukung beserta contoh penggunaannya

Jenis	Sintaks	Contoh	Elemen yang dipilih
<i>Tag</i>	<code>tag</code>	<code>p</code>	Semua <code><p></code>
<i>Class</i>	<code>.cls</code>	<code>.card</code>	Semua elemen ber-kelas <code>card</code>
<i>ID</i>	<code>#id</code>	<code>#main</code>	Elemen dengan <code>id="main"</code>
<i>Universal</i>	<code>*</code>	<code>*</code>	Semua elemen
<i>Child</i>	<code>A > B</code>	<code>ul > li</code>	<code></code> anak langsung <code></code>
<i>Descendant</i>	<code>A B</code>	<code>div p</code>	<code><p></code> di dalam <code><div></code>
<i>Adj. Sibling</i>	<code>A + B</code>	<code>h1 + p</code>	<code><p></code> tepat setelah <code><h1></code>
<i>Gen. Sibling</i>	<code>A ~ B</code>	<code>h1 ~ p</code>	Semua <code><p></code> setelah <code><h1></code>

2.4 Breadth-First Search (BFS)

Breadth-First Search (BFS) adalah algoritma penelusuran graf yang secara sistematis menjelajahi semua simpul pada sebuah komponen terhubung secara berlapis, dimulai dari simpul sumber dan meluas ke simpul-simpul tetangga sebelum melanjutkan ke simpul yang lebih jauh. Algoritma ini dianalisis secara formal oleh Cormen et al. sebagai salah satu algoritma dasar dalam teori graf [1].

2.4.1 Mekanisme Kerja

BFS menggunakan struktur data antrian (*queue*) yang bersifat FIFO (*First In, First Out*) untuk melacak simpul yang akan dikunjungi. Secara umum, untuk suatu pohon dengan akar r , BFS bekerja sebagai berikut:

1. Inisialisasi antrian dengan memasukkan simpul akar r .
2. Selama antrian tidak kosong:
 - (a) Keluarkan simpul u dari bagian depan antrian.
 - (b) Proses simpul u (periksa kecocokan selektor, catat ke log).
 - (c) Masukkan seluruh *child* dari u ke bagian belakang antrian secara berurutan.

Dalam konteks pohon DOM, penelusuran BFS berarti elemen-elemen diproses level per level — seluruh elemen pada kedalaman d diproses terlebih dahulu sebelum elemen-elemen pada kedalaman $d + 1$.

Algoritma 1 menunjukkan pseudocode BFS dalam konteks pohon DOM.

Algoritma 1 Breadth-First Search pada pohon DOM

```

1: Fungsi BFS(root, selector, limit)
2:   result  $\leftarrow$  []
3:   visited  $\leftarrow$  0
4:   Q  $\leftarrow$  antrian baru; ENQUEUE(Q, root)
5:   while Q tidak kosong and  $|result| < limit$  do
6:     u  $\leftarrow$  DEQUEUE(Q)
7:     visited  $\leftarrow$  visited + 1
8:     if MATCHESSELECTOR(u, selector) then
9:       APPEND(result, u)
10:    end if
11:    for each child  $\in$  u.children do
12:      ENQUEUE(Q, child)
13:    end for
14:  end while
15:  return result, visited
16: end Fungsi

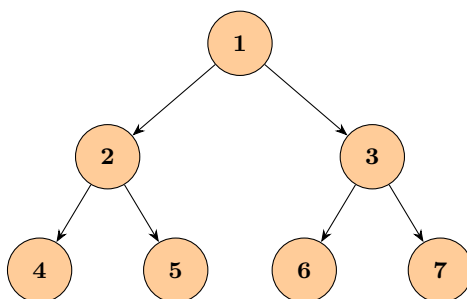
```

$\triangleright$ daftar elemen yang cocok

2.4.2 Sifat dan Kompleksitas

Pada pohon DOM dengan n simpul:

- **Kompleksitas Waktu:** $O(n)$ — setiap simpul dikunjungi paling banyak satu kali.
- **Kompleksitas Ruang:** $O(w)$ — di mana w adalah lebar maksimum pohon (*maximum width*), karena antrian pada suatu saat dapat berisi seluruh simpul pada satu level.
- **Hasil Terurut:** BFS menjamin bahwa elemen yang ditemukan terlebih dahulu adalah elemen yang paling dekat (secara kedalaman) dengan akar pohon DOM.



Gambar 2: Urutan penelusuran BFS (*level-order*): $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

2.5 Depth-First Search (DFS)

Depth-First Search (DFS) adalah algoritma penelusuran graf yang menjelajahi setiap simpul sejauh mungkin sepanjang setiap cabang sebelum melakukan *backtracking* ke cabang berikutnya. DFS pertama kali digunakan secara formal oleh Tarjan (1972) dalam konteks pencarian komponen terhubung kuat, dan dianalisis secara lengkap oleh Cormen et al. dalam kerangka teori graf umum [1].

2.5.1 Mekanisme Kerja

DFS dapat diimplementasikan secara rekursif (menggunakan *call stack* implisit) maupun iteratif menggunakan tumpukan (*stack*) eksplisit yang bersifat LIFO (*Last In, First Out*). Implementasi pada sistem ini menggunakan pendekatan iteratif dengan *stack* eksplisit untuk menghindari risiko *stack overflow* pada pohon DOM yang berkedalaman besar.

Penelusuran dimulai dengan memasukkan simpul akar ke dalam *stack*. Pada setiap iterasi, sejumlah simpul diambil dari ujung belakang *stack* (*last-in, first-out*), dicocokkan terhadap *selector*,

kemudian seluruh anak simpul tersebut didorong ke *stack* dalam urutan terbalik agar anak paling kiri berada di posisi paling atas dan diproses terlebih dahulu. Proses ini menghasilkan urutan penelusuran *pre-order* yang identik dengan implementasi rekursif.

Algoritma 2 menunjukkan pseudocode DFS iteratif berbasis *stack* dalam konteks pohon DOM.

Algoritma 2 Depth-First Search Iteratif pada pohon DOM

```

1: Prosedur DFS(root, selector, limit)
2:   stack  $\leftarrow$  [root]
3:   result  $\leftarrow$  []
4:   while stack  $\neq$   $\emptyset$  and (limit  $\leq$  0 or |result| < limit) do
5:     node  $\leftarrow$  POP(stack)
6:     if MATCHESSELECTOR(node, selector) then
7:       APPEND(result, node)
8:     end if
9:     for each child  $\in$  REVERSE(node.children) do
10:      PUSH(stack, child)
11:    end for
12:  end while
13:  return result
14: end Prosedur

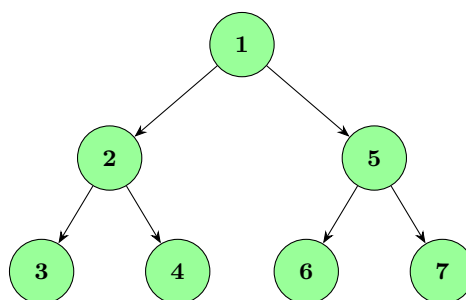
```

Anak-anak simpul didorong ke *stack* dalam urutan terbalik (baris 9) sehingga anak pertama (paling kiri) berada di posisi paling atas *stack* dan diproses pada iterasi berikutnya. Penelusuran berhenti lebih awal apabila jumlah kecocokan telah mencapai *limit* yang ditentukan.

2.5.2 Sifat dan Kompleksitas

Pada pohon DOM dengan n simpul:

- **Kompleksitas Waktu:** $O(n)$ — setiap simpul dikunjungi paling banyak satu kali.
- **Kompleksitas Ruang:** $O(h)$ — di mana h adalah kedalaman maksimum pohon (*height*), karena *stack* eksplisit menyimpan paling banyak satu jalur dari akar ke simpul daun pada satu waktu. DFS lebih hemat memori dibandingkan BFS pada pohon yang lebar namun tidak terlalu dalam.
- **Urutan Hasil:** Elemen yang ditemukan terlebih dahulu adalah elemen pada cabang paling kiri dan paling dalam.



Gambar 3: Urutan penelusuran DFS (*pre-order*): $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

2.5.3 Perbandingan BFS dan DFS

Tabel 2 merangkum perbedaan utama antara BFS dan DFS dalam konteks penelusuran pohon DOM.

Tabel 2: Perbandingan BFS dan DFS pada penelusuran pohon DOM

Aspek	BFS	DFS
Struktur data	Antrian (<i>queue</i>) FIFO	Tumpukan (<i>stack</i>) LIFO eksplisit
Urutan penelusuran	<i>Level-order</i>	<i>Pre-order</i>
Kompleksitas waktu	$O(n)$	$O(n)$
Kompleksitas ruang	$O(w)$, w = lebar maks.	$O(h)$, h = tinggi pohon
Keunggulan	Menemukan elemen terdekat	Hemat memori pada pohon lebar
Kelemahan	Boros memori pada pohon lebar	Tidak menjamin elemen terdekat

2.6 Lowest Common Ancestor dengan Binary Lifting

Lowest Common Ancestor (LCA) dari dua simpul u dan v pada sebuah pohon berakar adalah simpul terdalam yang merupakan leluhur (*ancestor*) dari keduanya. Simpul w disebut *ancestor* dari simpul v apabila w berada pada jalur dari v menuju akar pohon [5].

2.6.1 Definisi Formal

Diberikan pohon T berakar di r , LCA dari simpul u dan v didefinisikan sebagai:

$$\text{LCA}(u, v) = \arg \max_{w \in \text{Ancestors}(u) \cap \text{Ancestors}(v)} \text{depth}(w) \quad (1)$$

di mana $\text{depth}(w)$ menyatakan jarak (jumlah sisi) dari akar r ke simpul w .

2.6.2 Teknik Binary Lifting

Binary Lifting adalah teknik prakomputasi yang memungkinkan setiap query LCA dijawab dalam waktu $O(\log n)$ setelah dilakukan *preprocessing* $O(n \log n)$ [5]. Teknik ini memanfaatkan representasi biner dari bilangan bulat: setiap bilangan bulat positif k dapat dinyatakan sebagai jumlah perpangkatan dua, sehingga lompatan sebesar k tingkat ke atas dapat dilakukan dalam $O(\log k)$ langkah.

Tabel Leluhan (*Ancestor Table*):

Didefinisikan tabel $up[v][j]$ sebagai leluhan ke- 2^j dari simpul v :

$$up[v][0] = \text{parent}(v) \quad (2)$$

$$up[v][j] = up[up[v][j-1]][j-1], \quad j \geq 1 \quad (3)$$

Dengan $\text{LOG} = \lceil \log_2 n \rceil$, tabel berukuran $n \times \text{LOG}$ dapat dibangun dalam waktu $O(n \log n)$ menggunakan BFS atau DFS dari akar.

Contoh tabel $up[v][j]$ untuk pohon berikut:

$$1 \rightarrow 2 \rightarrow 4, \quad 2 \rightarrow 5, \quad 1 \rightarrow 3 \rightarrow 6, \quad 3 \rightarrow 7$$

Tabel 3: Contoh tabel $up[v][j]$ (*ancestor table*) untuk pohon dengan 7 simpul

Simpul v	$up[v][0]$ (leluhan ke- $2^0 = 1$)	$up[v][1]$ (leluhan ke- $2^1 = 2$)	$up[v][2]$
1	—	—	—
2	1	—	—
3	1	—	—
4	2	1	—
5	2	1	—
6	3	1	—
7	3	1	—

Algoritma Query LCA:

Algoritma 3 menunjukkan prosedur query LCA menggunakan tabel *Binary Lifting*.

Algoritma 3 Query LCA dengan Binary Lifting

```
1: Fungsi LCA( $u, v$ )
2:   if depth[ $u$ ] < depth[ $v$ ] then
3:     ( $u, v$ )  $\leftarrow$  ( $v, u$ )  $\triangleright$  pastikan depth[ $u$ ]  $\geq$  depth[ $v$ ]
4:   end if
5:   diff  $\leftarrow$  depth[ $u$ ] - depth[ $v$ ]
6:   for  $j \leftarrow \text{LOG} - 1$  downto 0 do
7:     if (diff  $\gg j$ ) and 1 = 1 then
8:        $u \leftarrow \text{up}[u][j]$   $\triangleright$  angkat  $u$  sebesar  $2^j$  tingkat
9:     end if
10:  end for
11:  if  $u = v$  then
12:    return  $u$   $\triangleright$  salah satu adalah leluhur yang lain
13:  end if
14:  for  $j \leftarrow \text{LOG} - 1$  downto 0 do
15:    if  $\text{up}[u][j] \neq \text{up}[v][j]$  then
16:       $u \leftarrow \text{up}[u][j]$ 
17:       $v \leftarrow \text{up}[v][j]$ 
18:    end if
19:  end for
20:  return  $\text{up}[u][0]$   $\triangleright$  leluhur bersama terdekat
21: end Fungsi
```

2.6.3 Kompleksitas dan Konteks Penerapan

- **Preprocessing:** $O(n \log n)$ waktu dan ruang untuk membangun tabel up .
- **Query:** $O(\log n)$ per query — jauh lebih efisien dari penelusuran naif $O(n)$.

Pada aplikasi tugas besar ini, tabel up dibangun sekali saja segera setelah pohon DOM selesai di-*parse*. Fitur LCA kemudian memungkinkan pengguna menemukan elemen HTML terdekat yang menjadi *container* bersama dari dua elemen yang dipilih — misalnya, menemukan `<div>` pembungkus bersama dari dua elemen hasil pencarian *CSS Selector*.

3 Aplikasi BFS dan DFS

3.1 Alur Kerja BFS pada Penelusuran DOM

Algoritma *Breadth-First Search* (BFS) menelusuri pohon DOM secara berlapis (*level-order*), memastikan bahwa seluruh simpul pada kedalaman d telah dikunjungi sebelum simpul pada kedalaman $d + 1$ mulai ditelusuri. Implementasi BFS pada sistem ini menggunakan *slice* Go sebagai antrian (*queue*) yang beroperasi secara FIFO (*First In, First Out*), dan memanfaatkan goroutine untuk melakukan pencocokan *CSS Selector* secara paralel pada setiap iterasi.

Inisialisasi Antrian

Penelusuran diawali dengan menempatkan simpul akar (*root*) ke dalam **frontier** yang berfungsi sebagai antrian.

```
frontier := []*model.DOMNode{root}
```

Seluruh proses eksplorasi berlangsung selama **frontier** tidak kosong dan jumlah kecocokan yang ditemukan belum mencapai batas **limit** yang ditentukan pengguna.

Pengambilan Node dari Antrian

Pada setiap iterasi, sejumlah node diambil dari bagian depan **frontier** sebanyak **batchSize**. Pengambilan dari ujung depan merupakan kunci dari semantik BFS, karena node yang paling awal dimasukkan (*first-in*) akan diproses terlebih dahulu (*first-out*).

```
batch = frontier[:batchSize]
frontier = frontier[batchSize:]
```

Ukuran *batch* ditentukan oleh nilai minimum antara **workerCount** (ditetapkan sebesar 4) dan jumlah node yang tersisa di **frontier**, sehingga setiap *batch* berisi paling banyak empat node.

Pencocokan Paralel dengan Goroutine

Setiap node dalam *batch* dikirimkan ke sebuah goroutine tersendiri untuk dilakukan pencocokan *CSS Selector* secara konkuren. Hasil pencocokan disimpan ke dalam *slice results* yang diindeks sesuai posisi node dalam *batch*, sehingga keterurutan tetap terjaga meskipun goroutine berjalan secara bersamaan.

```
results := make([]bool, batchSize)
var wg sync.WaitGroup
for i, node := range batch {
    wg.Add(1)
    go func(idx int, n *model.DOMNode) {
        defer wg.Done()
        results[idx] = selector.Matches(n, rawSel)
    }(i, node)
}
wg.Wait()
```

Seluruh goroutine ditunggu menggunakan **sync.WaitGroup** sebelum hasil diproses lebih lanjut, menjamin tidak ada kondisi balapan (*race condition*) pada tahap pemrosesan berikutnya.

Ekspansi Node ke Antrian

Setelah setiap node pada *batch* selesai diproses, seluruh anak dari node tersebut ditambahkan ke bagian belakang **frontier** sesuai urutan kiri ke kanan.

```
frontier = append(frontier, node.Children...)
```

Dengan strategi ini, anak-anak dari node pada kedalaman d selalu masuk ke antrian setelah seluruh node pada kedalaman d selesai diproses, sehingga penelusuran secara alami menghasilkan urutan *level-order*.

Pencatatan Log dan Penghentian Dini

Setiap node yang dikunjungi dicatat ke dalam log penelusuran beserta statusnya: "matched" apabila node memenuhi *CSS Selector* yang diberikan, atau "visiting" apabila tidak cocok.

```
logs = append(logs, model.TraversalStep{
    Step:    visitedCount,
    NodeID:  node.ID,
    Tag:     node.Tag,
    Status:  status,
})
```

Apabila jumlah kecocokan yang ditemukan telah memenuhi batas `limit`, penelusuran dihentikan secara langsung menggunakan pernyataan `goto Done` untuk keluar dari *loop* utama, menghindari komputasi yang tidak diperlukan pada sisa *frontier*.

3.2 Alur Kerja DFS pada Penelusuran DOM

Algoritma *Depth-First Search* (DFS) menelusuri pohon DOM dengan strategi mengeksplorasi satu cabang secara penuh hingga simpul daun (*leaf node*) sebelum berpindah ke cabang berikutnya. Implementasi DFS pada sistem ini menggunakan struktur data *stack* secara eksplisit yang direpresentasikan sebagai *slice* pada bahasa Go, menghindari penggunaan rekursi demi keamanan terhadap risiko *stack overflow* pada pohon DOM berkedalaman besar.

Inisialisasi Stack

Penelusuran diawali dengan menempatkan simpul akar (*root*) ke dalam *frontier* yang berfungsi sebagai *stack*.

```
frontier := []*model.DOMNode{root}
```

Seluruh proses eksplorasi berlangsung selama *frontier* tidak kosong dan jumlah kecocokan yang ditemukan belum mencapai batas `limit` yang ditentukan pengguna.

Pengambilan Node dari Stack

Pada setiap iterasi, sejumlah node diambil dari ujung belakang *frontier* sebanyak `batchSize`. Pengambilan dari ujung belakang merupakan kunci dari semantik DFS, karena node yang paling baru dimasukkan (*last-in*) akan diproses terlebih dahulu (*first-out*).

```
startIdx := len(frontier) - batchSize
batch = make([]*model.DOMNode, batchSize)
copy(batch, frontier[startIdx:])
for i, j := 0, len(batch)-1; i < j; i, j = i+1, j-1 {
    batch[i], batch[j] = batch[j], batch[i]
}
frontier = frontier[:startIdx]
```

Setelah disalin, urutan `batch` dibalik agar node yang berada paling atas *stack* diproses pada posisi pertama dalam `batch`. Perlu dicatat bahwa penggunaan `batchSize > 1` memperkenalkan limitasi, ketika beberapa node diambil sekaligus dalam satu *batch*, node-node tersebut dicocokkan secara paralel sebelum anak-anak dari node pertama dimasukkan ke *frontier*. Hal ini menyebabkan urutan kunjungan tidak sepenuhnya identik dengan DFS *pre-order* yang ketat, melainkan merupakan pendekatan *near-DFS* yang memprioritaskan eksplorasi mendalam namun dengan granularitas per *batch*.

Ekspansi Node ke Stack

Setelah seluruh node pada `batch` selesai diproses secara sekuensial, anak-anak dari setiap node didorong ke *frontier* dalam urutan terbalik, yaitu dari anak terakhir menuju anak pertama.

```
for j := len(node.Children) - 1; j >= 0; j-- {  
    frontier = append(frontier, node.Children[j])  
}
```

Dengan strategi ini, anak pertama (paling kiri) berada di posisi paling atas *stack* sehingga diprioritaskan pada iterasi berikutnya. Perilaku penelusuran yang dihasilkan tetap mendalam (*depth-prioritized*) dan secara praktis mendekati DFS, meskipun urutan eksak antar-simpul dalam satu *batch* tidak dijamin identik dengan DFS *pre-order* murni.

Pencatatan Log dan Penghentian Dini

Setiap node yang dikunjungi dicatat ke dalam log penelusuran beserta statusnya "matched" apabila node memenuhi *CSS selector* yang diberikan, atau "visiting" apabila tidak cocok.

```
logs = append(logs, model.TraversalStep{  
    Step:    visitedCount,  
    NodeID:  node.ID,  
    Tag:     node.Tag,  
    Status:  status,  
})
```

Apabila jumlah kecocokan yang ditemukan telah memenuhi batas *limit*, penelusuran dihentikan secara langsung menggunakan pernyataan `goto Done` untuk keluar dari *loop* utama, menghindari komputasi yang tidak diperlukan pada sisa *frontier*.

3.3 Mekanisme Pencocokan CSS Selector

Pencocokan CSS selector diimplementasikan pada paket `selector` melalui dua tahap utama, yaitu tokenisasi ekspresi selector menjadi urutan token, dan pencocokan rekursif berbasis kombinator terhadap struktur pohon DOM.

Tokenisasi Selector

Fungsi `tokenize` menguraikan string selector mentah menjadi urutan token yang merepresentasikan selektor tunggal dan kombinator di antaranya. Proses diawali dengan menyisipkan spasi di sekitar karakter kombinator eksplisit (`>`, `+`, `~`) agar dapat dikenali saat pemisahan menggunakan `strings.Fields`.

```
s = strings.ReplaceAll(s, ">", " > ")  
s = strings.ReplaceAll(s, "+", " + ")  
s = strings.ReplaceAll(s, "~", " ~ ")
```

Setelah pemisahan, apabila dua token selektor berurutan tanpa kombinator eksplisit di antaranya, secara otomatis disisipkan token spasi (" ") yang merepresentasikan kombinator *descendant*. Dengan demikian, seluruh relasi antar selektor, baik yang ditulis eksplisit maupun implisit, terwakili secara seragam dalam bentuk token.

Pencocokan Rekursif dengan Kombinator

Fungsi `matchCombinator` mencocokkan node saat ini terhadap token paling kanan dari ekspresi selector, kemudian secara rekursif menelusuri pohon ke atas mengikuti jenis kombinator yang mendahului token tersebut. Empat jenis kombinator yang didukung beserta implementasinya dijelaskan sebagai berikut.

Child Combinator (>) Mencocokkan node apabila *parent* langsungnya memenuhi ekspresi bagian sebelumnya. Relasi yang diuji bersifat langsung satu tingkat ke atas.

```
case ">":  
    return matchCombinator(node.Parent, parts, prevIdx)
```

Descendant Combinator (spasi) Menelusuri seluruh leluhur (*ancestor*) node saat ini secara berurutan ke atas hingga menemukan satu yang memenuhi ekspresi bagian sebelumnya. Tidak dibatasi hanya satu tingkat sehingga mencakup seluruh kedalaman pohon di atas node saat ini.

```
case " ":
    curr := node.Parent
    for curr != nil {
        if matchCombinator(curr, parts, prevIdx) {
            return true
        }
        curr = curr.Parent
    }
```

Adjacent Sibling Combinator (+) Mencocokkan node apabila *sibling* yang tepat berada satu posisi sebelumnya memenuhi ekspresi bagian sebelumnya. Fungsi bantu `getPrevSibling` digunakan untuk mendapatkan saudara sebelumnya dengan menelusuri daftar anak dari *parent* yang sama.

```
case "+":
    prev := getPrevSibling(node)
    return matchCombinator(prev, parts, prevIdx)
```

General Sibling Combinator (~) Menelusuri semua *sibling* yang berada sebelum node saat ini, tidak harus yang paling berdekatan, hingga menemukan satu yang memenuhi ekspresi bagian sebelumnya.

```
case "~":
    curr := getPrevSibling(node)
    for curr != nil {
        if matchCombinator(curr, parts, prevIdx) {
            return true
        }
        curr = getPrevSibling(curr)
    }
```

Pencocokan Selektor Tunggal

Fungsi `matchSingle` mencocokkan sebuah node terhadap satu token selektor tanpa kombinator. Pemeriksaan dilakukan secara berurutan berdasarkan awalan token dengan prioritas sebagai berikut.

- **Universal Selector (*)**: selalu mengembalikan `true` untuk semua node tanpa syarat.
- **Attribute Selector ([attr] atau [attr=val])**: memeriksa keberadaan atribut pada `node.Attributes`, atau mencocokkan nilainya secara tepat apabila operator `=` disertakan.
- **ID Selector (#id)**: mencocokkan nilai `node.IDAttr` dengan identifikasi yang diberikan, dengan dukungan opsional prefiks nama tag sebelum karakter `#`.
- **Class Selector (.class)**: memeriksa apakah nama kelas yang diberikan terdapat dalam `slice node.Classes`, dengan dukungan opsional prefiks nama tag sebelum karakter `..`
- **Tag Selector**: mencocokkan `node.Tag` secara langsung apabila tidak ada awalan khusus yang ditemukan pada token.

3.4 Paralelisasi Pencocokan dengan Goroutine

Guna meningkatkan efisiensi komputasi pada pohon DOM berukuran besar, implementasi memanfaatkan *goroutine* pada bahasa Go sebagai mekanisme paralelisasi. Paralelisasi diterapkan secara terisolasi pada tahap pencocokan CSS selector, sementara urutan eksplorasi node tetap mengikuti semantik BFS maupun DFS secara ketat dan deterministik.

Strategi Batch Parallelism

Pendekatan yang digunakan adalah *batch parallelism*, pada setiap iterasi penelusuran, sejumlah node yang telah diambil dari **frontier**, maksimum sebanyak **workerCount = 4**, dicocokkan terhadap CSS selector secara bersamaan menggunakan *goroutine* yang terpisah untuk setiap node.

```
results := make([]bool, batchSize)
var wg sync.WaitGroup

for i, node := range batch {
    wg.Add(1)
    go func(idx int, n *model.DOMNode) {
        defer wg.Done()
        results[idx] = selector.Matches(n, rawSel)
    }(i, node)
}

wg.Wait()
```

Setiap *goroutine* menerima indeks dan node yang akan diproses sebagai argumen tersendiri sehingga tidak terjadi *data race* pada pembacaan data node. Hasil pencocokan disimpan pada *slice results* di indeks yang telah ditentukan sebelumnya, sehingga penulisan antar *goroutine* bersifat *index-disjoint* dan tidak memerlukan mekanisme penguncian (*mutex*). Sinkronisasi dilakukan menggunakan `sync.WaitGroup`, di mana pemanggil menunggu seluruh *goroutine* selesai melalui `wg.Wait()` sebelum melanjutkan ke tahap pengumpulan hasil.

Menjaga Konsistensi Urutan BFS dan DFS

Meskipun pencocokan dilakukan secara paralel, urutan eksplorasi pohon DOM tetap deterministik dan sesuai dengan algoritma yang dipilih. Hal ini dijamin melalui pemisahan yang tegas antara fase paralel (pencocokan) dan fase sekuensial (manajemen **frontier** dan pencatatan hasil).

Pada mode BFS, node diambil dari ujung depan **frontier** (*first-in, first-out*), memastikan node pada level yang lebih dangkal selalu diproses sebelum node pada level yang lebih dalam.

```
batch = frontier[:batchSize]
frontier = frontier[batchSize:]
```

Pada mode DFS, node diambil dari ujung belakang **frontier** (*last-in, first-out*), dan anak-anak node dimasukkan kembali ke **frontier** dalam urutan terbalik agar anak pertama berada di posisi paling atas *stack* untuk diproses lebih dahulu.

```
startIdx := len(frontier) - batchSize
batch = make([]*model.DOMNode, batchSize)
copy(batch, frontier[startIdx:])
for i, j := 0, len(batch)-1; i < j; i, j = i+1, j-1 {
    batch[i], batch[j] = batch[j], batch[i]
}
frontier = frontier[:startIdx]
```

Ekspansi anak ke **frontier** baru terjadi secara sekuensial setelah seluruh *goroutine* selesai dan hasil dikumpulkan. Dengan demikian, paralelisasi sepenuhnya terisolasi pada operasi pencocokan yang bersifat *stateless*, tidak mengubah struktur pohon maupun **frontier**, sehingga tidak mengganggu determinisme urutan penelusuran.

Penghentian Dini

Apabila jumlah kecocokan yang ditemukan telah mencapai batas **limit** yang ditentukan pengguna, penelusuran dihentikan segera menggunakan `goto Done` tanpa memproses sisa elemen pada **batch** maupun **frontier** yang tersisa.

```
if limit > 0 && len(matches) >= limit {  
    goto Done  
}
```

Mekanisme ini memastikan bahwa paralelisasi tidak mengorbankan efisiensi pada kasus di mana hasil yang dibutuhkan sudah terpenuhi sebelum seluruh pohon dieksplorasi.

3.5 Animasi Traversal

Animasi penelusuran pada sistem ini diimplementasikan sepenuhnya pada sisi *frontend* menggunakan mekanisme *playback* berbasis log penelusuran, tanpa bergantung pada aliran data *real-time* dari *backend*.

3.5.1 Alur Umum

Setelah pengguna menekan tombol *Start Search*, *frontend* mengirimkan permintaan `POST /api/traverse` ke *backend*. *Backend* menjalankan BFS atau DFS secara penuh dan mengembalikan seluruh log penelusuran sekaligus dalam satu respons JSON. Respons ini mencakup pohon DOM, daftar *match*, dan *slice* dari `TraversalStep` yang merekam setiap kunjungan node beserta statusnya.

3.5.2 State Management pada useTraversal

Custom hook useTraversal bertanggung jawab mengelola seluruh *state* animasi. Setelah respons diterima, hook menyimpan log lengkap ke dalam `fullData` dan menginisialisasi `currentStep` ke nilai total langkah (sehingga seluruh log langsung terlihat sebelum animasi diputar ulang). Fungsi `run` juga memanggil `setCurrentStep` dengan nilai penuh setelah fetch selesai.

Untuk animasi, hook menggunakan `useEffect` yang menjalankan `setInterval` selama `isPlaying` bernilai `true`. Setiap interval, `currentStep` dinaikkan sebesar `batchSize` sesuai preset kecepatan yang dipilih.

```
interval = window.setInterval(() => {  
    setCurrentStep((prev) => {  
        const next = prev + batchSize;  
        return next >= fullData.log.length  
            ? fullData.log.length  
            : next;  
    });  
}, playbackSpeed);
```

3.5.3 Komputasi animatedData

Nilai `animatedData` dihitung ulang setiap kali `currentStep` berubah menggunakan `useMemo`. Log di-*slice* hingga indeks `currentStep`, kemudian ID-ID yang terpengaruh diekstrak:

```
const slicedLog = fullData.log.slice(0, currentStep);  
const visitedIds = slicedLog.map((l) => l.nodeId);  
const visitedSet = new Set(visitedIds);  
const currentMatchedIds = fullData.matchedIds  
    .filter((id) => visitedSet.has(id));  
const currentPathIds = computePathIds(  
    fullData.tree, currentMatchedIds);
```

Nilai ini kemudian diteruskan ke `VisualizerCanvas` sebagai prop `visitedIds`, `matchedIds`, dan `pathIds`, sehingga kanvas menyoroti node yang sesuai.

3.5.4 Komponen PlaybackControls

Komponen `PlaybackControls` menyediakan antarmuka kontrol animasi: tombol *play/pause*, tombol *replay*, *slider* untuk *scrubbing* posisi langkah secara manual, dan pemilihan preset kecepatan (Slow, Normal, Fast, Instant). Preset kecepatan mendefinisikan interval *timer* (ms) dan ukuran lompatan langkah (*batch size*) per *tick*.

Tabel 4: Preset kecepatan animasi pada komponen `PlaybackControls`

Label	Interval (ms)	Langkah per tick
Slow	500	1
Normal	200	1
Fast	16	5
Instant	16	25

3.6 Penerapan LCA Binary Lifting pada DOM

Fitur *Lowest Common Ancestor* (LCA) pada aplikasi ini memanfaatkan teknik *Binary Lifting* yang diimplementasikan pada paket `lca` di *backend*.

3.6.1 Pembangunan Tabel (Build)

Tabel *ancestor* dibangun oleh fungsi `Build(root *model.DOMNode)` segera setelah pohon DOM selesai diparse, baik melalui endpoint `/api/scrape` maupun `/api/traverse`. Fungsi ini menggunakan *stack* eksplisit (bukan rekursi) untuk menelusuri seluruh pohon dan mengisi tiga struktur data utama:

- **depth**: peta dari ID node ke nilai kedalamannya.
- **nodes**: peta dari ID node ke pointer `DOMNode`.
- **up[k]**: tabel peta ID node ke ID leluhur ke- 2^k -nya, dengan $\text{LOG} = 18$ (mendukung pohon hingga $2^{18} = 262.144$ node).

Setelah DFS selesai, *binary lifting* dilakukan untuk mengisi `up[1]` hingga `up[LOG-1]`:

```
for k := 1; k < LOG; k++ {
  for id := range t.nodes {
    mid := t.up[k-1][id]
    if mid == "" {
      t.up[k][id] = ""
    } else {
      t.up[k][id] = t.up[k-1][mid]
    }
  }
}
```

3.6.2 Penyimpanan pada Store

Tabel LCA disimpan bersama pohon DOM pada *in-memory store* (`internal/store/tree_store.go`). Fungsi `store.SetTree(t)` secara atomik memperbarui pohon dan membangun ulang tabel LCA menggunakan *mutex* sehingga aman diakses secara konkuren.

3.6.3 Query LCA via API

Handler `POST /api/lca` menerima dua ID node (**node-a** dan **node-b**) berdasarkan **node-d** pada DOM Tree yang sudah terbentuk, kemudian memanggil `table.Query(a,b)`, dan mengembalikan node LCA beserta kedalaman masing-masing node input. Pada *frontend*, komponen `LCAPanel` mengirimkan permintaan ini dan hasilnya ditampilkan pada `StatsFooter`: nama tag dan ID node LCA, serta kedalaman node A dan B.

4 Implementasi dan Pengujian

4.1 Lingkungan Pengembangan

Aplikasi DOM Tree Visualizer dikembangkan pada lingkungan lokal dengan sistem operasi Windows dan Linux, menggunakan seperangkat perangkat lunak yang dirangkum pada tabel berikut.

Komponen	Versi	Keterangan
Go	1.25+	Bahasa pemrograman utama untuk <i>backend</i> . Digunakan bersama modul github.com/gin-gonic/gin sebagai kerangka kerja HTTP dan golang.org/x/net/html sebagai <i>parser</i> HTML.
Bun	1.x	<i>Package manager</i> dan <i>runtime</i> JavaScript yang digunakan untuk instalasi dependensi serta menjalankan <i>development server frontend</i> .
Node.js	20.x+	Dibutuhkan oleh Vite sebagai <i>build tool frontend</i> .
Docker	24.x	Digunakan untuk mengemas <i>backend</i> dan <i>frontend</i> ke dalam kontainer yang dapat dijalankan secara konsisten di berbagai lingkungan.
Docker Compose	v2.x	Mengorkestrasi dua layanan kontainer (<i>backend</i> dan <i>frontend</i>) sekaligus dengan satu perintah <code>docker compose up --build</code> .
Git	—	Sistem pengendalian versi yang digunakan untuk mengelola riwayat perubahan kode sumber selama pengembangan.

Tabel 5: Perangkat lunak yang digunakan dalam pengembangan

Selama pengembangan lokal, *backend* dijalankan pada port 8080 menggunakan perintah `go run ./cmd/main.go` dari direktori `backend/`, sementara *frontend* dijalankan pada port 3000 menggunakan perintah `bun run dev` dari direktori `frontend/`. Seluruh permintaan HTTP dari *frontend* ke awalan `/api/` diteruskan secara otomatis ke *backend* melalui konfigurasi *proxy* pada Vite, sehingga tidak diperlukan konfigurasi CORS tambahan selama pengembangan.

Variabel lingkungan dikelola melalui berkas `.env` yang disalin dari `.env.example`. Dua variabel utama yang digunakan adalah `PORT` untuk menentukan port *backend*, dan `ALLOWED_ORIGIN` untuk mengatur asal permintaan yang diizinkan oleh *middleware* CORS pada Gin. Pada sisi *frontend*, variabel `VITE_API_BASE_URL` didefinisikan pada `.env.example` dan `docker-compose.yml` sebagai URL dasar *backend*. Namun demikian, modul `api/client.ts` membaca variabel `VITE_API_URL` secara langsung — perlu dipastikan konsistensi nama variabel ini antara berkas `.env` dan konfigurasi Vite agar pemanggilan HTTP berjalan dengan benar di lingkungan produksi.

Untuk keperluan *deployment* produksi, aplikasi dikemas menggunakan Docker dengan dua *image* terpisah. *Image backend* dibangun dari `golang:1.24-alpine` menggunakan pola *multi-stage build* yang menghasilkan *binary* statis `server`. *Image frontend* dibangun dari `oven/bun:1` untuk tahap *build*, kemudian berkas statis hasilnya disalin ke *image nginx:alpine* yang bertugas menyajikan *frontend* sekaligus bertindak sebagai *reverse proxy* ke *backend*.

4.2 Struktur Program

Repositori proyek diorganisasikan menjadi dua direktori utama, `backend/` dan `frontend/`, yang masing-masing merupakan modul independen dengan dependensi dan konfigurasi *build* tersendiri. Pada tingkat paling atas terdapat berkas konfigurasi bersama yang mengatur orkestrasi kontainer dan pengelolaan variabel lingkungan.

Struktur Direktori Backend

Direktori `backend/` mengikuti konvensi standar proyek Go dengan memisahkan kode *entry point* pada `cmd/` dan seluruh logika internal pada `internal/`.

```
backend/  
├── cmd/  
│   └── main.go ..... Entry point: inisialisasi Gin dan pendaftaran route
```

```

internal/
├── handler/
│   ├── scrape.go ..... Handler POST /api/scrape
│   ├── traverse.go ..... Handler POST /api/traverse
│   ├── traverse_stream.go ..... Handler GET /api/traverse/stream (SSE)
│   └── lca.go ..... Handler POST /api/lca
├── model/
│   └── types.go ..... Definisi struct DOMNode, TraverseResponse, LCAResult, dst.
├── parser/
│   └── parser.go ..... Pengurai HTML menjadi pohon DOMNode
├── scraper/
│   └── scraper.go ..... Pengambil HTML dari URL menggunakan HTTP client
├── selector/
│   └── selector.go ..... Tokenisasi dan pencocokan CSS Selector
├── store/
│   └── tree_store.go ..... In-memory store pohon DOM dan tabel LCA
├── traversal/
│   ├── concurrent.go ..... Implementasi ConcurrentBFS dan ConcurrentDFS
│   └── logger.go ..... Pencatat log penelusuran ke berkas JSON
├── lca/
│   └── lca.go ..... Binary Lifting LCA: Build dan Query
├── logs/ ..... Direktori keluaran log penelusuran (dibuat saat runtime)
├── go.mod
├── go.sum
└── Dockerfile

```

Setiap paket pada `internal/` memiliki tanggung jawab tunggal dan tidak saling mengimpor secara siklik. Paket `model` menjadi titik temu antar paket karena mendefinisikan tipe data bersama yang digunakan oleh `handler`, `traversal`, `parser`, `store`, dan `lca`. Paket `store` berfungsi sebagai lapisan penyimpanan global yang menghubungkan `handler` dengan `lca` tanpa ketergantungan langsung di antara keduanya.

Struktur Direktori Frontend

Direktori `frontend/` mengikuti konvensi proyek Vite dengan React, di mana seluruh kode sumber berada pada `src/` dan diorganisasikan berdasarkan perannya.

```

frontend/
├── src/
│   ├── api/
│   │   └── client.ts ..... Enkapsulasi pemanggilan HTTP ke backend
│   ├── components/
│   │   ├── ControlPanel.tsx ..... Formulir masukan penelusuran
│   │   ├── VisualizerCanvas.tsx ..... Kanvas graf DOM interaktif
│   │   ├── LogTerminal.tsx ..... Terminal log penelusuran
│   │   ├── PlaybackControls.tsx ..... Kontrol animasi langkah per langkah
│   │   ├── LCAPanel.tsx ..... Formulir dan hasil query LCA
│   │   ├── StatsFooter.tsx ..... Footer metrik kinerja
│   │   ├── LoadingOverlay.tsx ..... Indikator pemuatan
│   │   ├── Navbar.tsx ..... Navigasi antar halaman
│   │   ├── LandingPage.tsx ..... Halaman utama
│   │   └── HeroTree.tsx ..... Ilustrasi pohon DOM pada halaman utama
│   ├── hooks/
│   │   └── useTraversal.ts ..... Custom hook manajemen state penelusuran dan animasi
│   ├── types/
│   │   └── api.ts ..... Definisi tipe TypeScript untuk request dan response API
│   ├── App.tsx ..... Root layout dan routing antar halaman
│   ├── main.tsx ..... Entry point React
│   ├── index.css ..... Gaya global dan konfigurasi Tailwind CSS
│   ├── public/ ..... Aset statis
│   └── nginx.conf ..... Konfigurasi Nginx untuk produksi

```

```
├─ Dockerfile
├─ package.json
├─ vite.config.ts
└─ tailwind.config.js
```

Berkas Konfigurasi Tingkat Atas

Pada direktori akar repositori terdapat beberapa berkas konfigurasi yang mengatur lingkungan secara keseluruhan.

Berkas	Fungsi
<code>docker-compose.yml</code>	Mendefinisikan dua layanan kontainer (backend dan frontend) beserta konfigurasi port dan variabel lingkungan masing-masing.
<code>.env.example</code>	Templat variabel lingkungan yang perlu disalin menjadi <code>.env</code> sebelum menjalankan aplikasi. Memuat <code>PORT</code> , <code>ALLOWED_ORIGIN</code> , dan <code>VITE_API_BASE_URL</code> .
<code>.gitignore</code>	Mengecualikan berkas yang tidak perlu dilacak oleh Git, termasuk <code>node_modules/</code> , <code>dist/</code> , <code>.env</code> , dan direktori <code>backend/logs/</code> .

Tabel 6: Berkas konfigurasi pada direktori akar repositori

4.3 Implementasi Backend

Bagian ini menjelaskan arsitektur dan integrasi komponen pada sisi *backend* yang dibangun menggunakan bahasa pemrograman Go. Fokus utama dari implementasi ini adalah mengoordinasikan berbagai modul internal untuk melayani permintaan dari *frontend* secara efisien.

4.3.1 Arsitektur dan Integrasi Komponen

Backend aplikasi ini dirancang dengan arsitektur modular yang membagi fungsionalitas ke dalam beberapa paket utama di dalam direktori `internal/`. Integrasi antar komponen dikelola melalui *entry point* utama pada `cmd/main.go`.

- **Layer Handler** (`internal/handler/`): Berfungsi sebagai pintu masuk (*entry point*) API. Paket ini bertanggung jawab untuk melakukan *parsing* terhadap *payload* JSON dari *frontend*, memvalidasi parameter, dan memanggil logika bisnis yang sesuai.
- **Layer Parser** (`internal/parser/`): Merupakan jembatan antara data mentah (HTML) dan struktur data aplikasi. Komponen ini mengonversi *string* HTML menjadi struktur *N-ary Tree* (DOMNode).
- **Layer Traversal dan Selector** (`internal/traversal/`, `internal/selector/`): Handler akan menginstruksikan modul *traversal* untuk menjalankan algoritma pencarian. Modul ini terintegrasi erat dengan modul *selector* yang bertugas mengevaluasi kecocokan setiap *node* terhadap kueri CSS *selector*.
- **Layer Logger** (`internal/traversal/logger.go`): Selama proses algoritma berjalan, setiap langkah (kunjungan *node*) dicatat ke dalam log penelusuran untuk kemudian dikirimkan sekaligus sebagai data animasi di sisi klien.

4.3.2 Alur Pemrosesan Query

Ketika *backend* menerima *query* dari *frontend*, terdapat alur kerja terintegrasi sebagai berikut:

1. **Penerimaan Request**: Server menerima permintaan HTTP POST melalui *endpoint* `/api/traverse`. Handler mengekstrak parameter seperti URL atau *raw HTML*, algoritma yang dipilih (BFS/DFS), dan CSS *selector*.

2. **Konversi HTML ke Struktur Tree:** Jika input berupa URL, modul *scraper* akan mengambil isi halaman terlebih dahulu. Selanjutnya, modul *parser* memproses *string* HTML tersebut. Modul ini melakukan *mapping* atribut HTML (seperti *id*, *class*, dan *tag*) ke dalam objek *DOMNode* dan menyusunnya secara hierarkis dalam struktur pohon.
3. **Eksekusi Algoritma dan Pencocokan:** Struktur pohon yang telah terbentuk kemudian dilewatkan ke fungsi *traversal*. Di sini, integrasi terjadi antara mesin penelusuran (BFS/DFS) dengan mesin pencocokan (*selector engine*). Setiap kali mesin penelusuran mengunjungi sebuah *node*, ia akan memanggil modul *selector* untuk menentukan apakah *node* tersebut adalah target yang dicari.
4. **Logging dan Manajemen Hasil:** Hasil pencocokan dikumpulkan, dan setiap langkah penelusuran dicatat oleh modul *logger*. Jika kueri adalah kueri LCA (*Lowest Common Ancestor*), sistem akan merujuk pada *tree* yang tersimpan di dalam memori (*Tree Store*) untuk mencari leluhur bersama tercepat tanpa perlu melakukan *parsing* ulang.
5. **Pengembalian Response:** Data hasil akhir yang berisi struktur pohon lengkap, daftar *node* yang cocok, dan log penelusuran di-serialisasi menjadi format JSON dan dikirimkan kembali ke *frontend* untuk divisualisasikan.

4.3.3 Integrasi Frontend-Backend

Komunikasi antara kedua sisi dilakukan melalui protokol HTTP dengan pertukaran data berbasis JSON. *Backend* dikonfigurasi untuk menangani *Cross-Origin Resource Sharing* (CORS) agar dapat menerima permintaan dari domain *frontend*. Selain itu, integrasi ini juga menyediakan *endpoint* SSE melalui `traverse.stream.go` sebagai kapabilitas *streaming* langkah penelusuran secara *real-time*. Pada implementasi *frontend* yang digunakan saat ini, animasi dijalankan secara *client-side* melalui pemutaran ulang log dari respons `POST /api/traverse`.

4.4 Implementasi Frontend

Bagian ini menjelaskan arsitektur dan integrasi komponen pada sisi *frontend* yang dibangun menggunakan React 19 dengan TypeScript dan Vite. Fokus utama dari implementasi ini adalah mengoordinasikan berbagai komponen antarmuka pengguna serta mengelola alur data dari pemanggilan API hingga visualisasi animasi penelusuran.

4.4.1 Arsitektur dan Integrasi Komponen

Frontend aplikasi ini dirancang dengan arsitektur berbasis komponen yang membagi fungsionalitas ke dalam beberapa lapisan terpisah. Integrasi antar komponen dikelola melalui *root layout* pada `App.tsx` dan *custom hook* `useTraversal` sebagai pusat manajemen *state*.

- **Layer API Client** (`src/api/client.ts`): Berfungsi sebagai lapisan komunikasi tunggal antara *frontend* dan *backend*. Paket ini mengenkapsulasi seluruh pemanggilan HTTP melalui fungsi generik `post<T>` menggunakan *Fetch API* bawaan *browser*, sehingga komponen lain tidak perlu mengetahui detail protokol komunikasi secara langsung.
- **Layer Hook** (`src/hooks/useTraversal.ts`): Merupakan pusat manajemen *state* seluruh siklus hidup penelusuran. *Hook* ini bertanggung jawab atas pemanggilan API, transformasi respons, komputasi jalur visualisasi, serta pengendalian mesin animasi langkah per langkah.
- **Layer Komponen** (`src/components/`): Setiap komponen memiliki tanggung jawab tunggal dan menerima data dari *hook* `useTraversal` melalui *props*. Komponen-komponen ini meliputi `VisualizerCanvas` untuk graf interaktif, `ControlPanel` untuk formulir masukan, `LogTerminal` untuk log penelusuran, `PlaybackControls` untuk kontrol animasi, serta `LCAPanel` dan `StatsFooter` untuk fitur LCA dan metrik kinerja.
- **Layer Routing** (`App.tsx`): Mengelola navigasi antara halaman `LandingPage` dan halaman visualisasi utama menggunakan *state* sederhana yang dipersistensikan ke `localStorage`, tanpa ketergantungan pada *library* routing eksternal.

4.4.2 Alur Pemrosesan dan Visualisasi

Ketika pengguna menginisiasi penelusuran dari *frontend*, terdapat alur kerja terintegrasi sebagai berikut:

1. **Penerimaan Masukan:** Komponen `ControlPanel` mengumpulkan parameter penelusuran dari pengguna, yaitu sumber HTML (URL atau *raw HTML*), pilihan algoritma (BFS atau DFS), CSS *selector*, serta mode tampilan (*Top N* atau semua kemunculan). Pergantian mode masukan mengosongkan nilai sebelumnya untuk menghindari ambiguitas.
2. **Pemanggilan API dan Transformasi Respons:** *Hook* `useTraversal` memanggil `api.traverse` melalui *layer* `API client`. Respons mentah dari *backend* kemudian ditransformasi menggunakan fungsi `transformResponse`, yang mencakup konversi nama *field* dari format *snake-case* Go ke *camelCase* TypeScript, serta komputasi `pathIds` yaitu himpunan seluruh ID simpul yang berada pada jalur dari akar ke setiap simpul yang cocok.
3. **Manajemen State Dua Level:** Hasil transformasi disimpan ke `fullData` yang berisi data penelusuran lengkap. Secara terpisah, `animatedData` dihitung ulang setiap kali posisi langkah animasi (`currentStep`) berubah menggunakan `useMemo`, menghasilkan irisan `visitedIds`, `matchedIds`, dan `pathIds` yang hanya mencakup simpul-simpul yang telah dikunjungi hingga langkah saat ini.
4. **Pengendalian Animasi:** Mesin animasi dikendalikan oleh *state* `currentStep`, `isPlaying`, dan `playbackSpeed`. Sebuah `useEffect` menjalankan `setInterval` selama animasi berjalan, menaikkan `currentStep` sebanyak `batchSize` per *tick* sesuai preset kecepatan yang dipilih. Tersedia empat preset: *Slow* (500 ms, 1 langkah), *Normal* (200 ms, 1 langkah), *Fast* (16 ms, 5 langkah), dan *Instant* (16 ms, 25 langkah).
5. **Rendering Visualisasi:** Komponen `VisualizerCanvas` menerima `animatedData` dan merender pohon DOM sebagai graf interaktif menggunakan pustaka `@xyflow/react`. Setiap simpul dirender dengan kelas CSS yang mencerminkan statusnya (`matched`, `path`, `visited`, atau `default`), sementara *edge* antara dua simpul dianimasikan hanya apabila keduanya berstatus `matched` atau `path` secara bersamaan.

4.4.3 Integrasi Komponen Pendukung

Selain alur penelusuran utama, terdapat beberapa komponen pendukung yang beroperasi secara terintegrasi untuk melengkapi fungsionalitas aplikasi.

Komponen `LogTerminal` menampilkan log penelusuran dalam format terminal dengan tiga kondisi tampilan: kondisi diam saat belum ada penelusuran, kondisi memuat saat penelusuran sedang berjalan, dan kondisi aktif saat log tersedia. *Auto-scroll* ke bawah diimplementasikan menggunakan `useRef` yang dipicu setiap kali log diperbarui.

Komponen `PlaybackControls` menyediakan antarmuka kontrol animasi yang hanya dirender apabila data penelusuran tersedia. Komponen ini menampilkan tombol *replay*, tombol *play/pause*, *slider* posisi langkah, persentase kemajuan, serta menu pilihan kecepatan animasi.

Komponen `LCAPanel` beroperasi secara independen dari alur penelusuran utama dengan memanggil `api.lca` secara langsung tanpa melalui *hook* `useTraversal`. Hasil *query* LCA diteruskan ke komponen induk melalui *callback* `onResult` untuk ditampilkan pada `StatsFooter` bersama dengan lima metrik kinerja lainnya: kedalaman maksimum pohon, jumlah simpul yang dikunjungi, jumlah hasil yang cocok, waktu pencarian dalam milidetik, serta simpul LCA beserta kedalaman masing-masing.

4.5 Pengujian

1. Test Case 1

Input	
Sumber	Raw HTML
Konten HTML	<pre> <html> <head><title>Test</title></head> <body> <p>Paragraf pertama</p> <p>Paragraf kedua</p> <div> <p>Paragraf dalam div</p> </div> </body> </html> </pre>
Algoritma	BFS
Opsi Pencarian	Semua kemunculan
CSS Selector	p
Output	
Hasil Visualisasi	

Tabel 7: Test Case 1

Analisis: Selektor `p` memilih seluruh elemen `<p>` tanpa memandang posisinya. BFS menemukan dua `<p>` di bawah `<body>` terlebih dahulu sebelum `<p>` di dalam `<div>`, menghasilkan tiga kecocokan.

2. Test Case 2

Input	
Sumber	Raw HTML
Konten HTML	<pre> <html> <body> <div class="container"> <section> <p class="text">Dalam section</p> </section> <p class="text">Langsung dalam div</p> </div> <p class="text">Di luar div</p> </body> </html> </pre>
Algoritma	DFS
Opsi Pencarian	Semua kemunculan
CSS Selector	.container .text
Output	
Hasil Visualisasi	

Tabel 8: Test Case 2

Analisis: Selektor `.container .text` memilih elemen berkelas `text` yang merupakan keturunan `.container`. Elemen `<p>` di luar `.container` tidak terpilih, menghasilkan dua kecocokan.

3. Test Case 3

Input	
Sumber	Raw HTML
Konten HTML	<pre> <html> <body> <nav id="menu"> Home About Ini bukan child langsung ul </nav> Luar nav </body> </html> </pre>
Algoritma	BFS
Opsi Pencarian	Top N kemunculan, $N = 10$
CSS Selector	#menu > ul > li
Output	
Hasil Visualisasi	

Tabel 9: Test Case 3

Analisis: Selektor #menu > ul > li hanya memilih yang merupakan *child* langsung di dalam #menu. Meskipun $N = 10$, hasil aktual hanya dua kecocokan, membuktikan aplikasi tidak galat saat N melebihi jumlah kecocokan yang ada.

4. Test Case 4

Input	
Sumber	URL
URL	https://example.com
Algoritma	BFS
Opsi Pencarian	Semua kemunculan
CSS Selector	div > p
Output	
Hasil Visualisasi	

Tabel 10: Test Case 4

Analisis: Selektor `div > p` memilih elemen `<p>` yang merupakan *child* langsung `<div>`. Dari 11 node yang dikunjungi pada pohon berkedalaman maksimum 4, ditemukan 2 kecocokan yakni seluruh `<p>` yang menjadi *child* langsung satu-satunya `<div>` pada halaman tersebut.

5. Test Case 5

Input	
Sumber	URL
URL	https://www.geeksforgeeks.org/
Algoritma	DFS
Opsi Pencarian	Top N kemunculan, $N = 5$
CSS Selector	div a
Output	
Hasil Visualisasi	

Tabel 11: Test Case 5

Analisis: Selektor `div a` memilih seluruh `<a>` keturunan `<div>` pada kedalaman berapapun. DFS berhasil menemukan 5 kecocokan setelah mengunjungi 53 node dari pohon berkedalaman maksimum 17, dengan waktu eksekusi 0,11 ms. Penelusuran berhenti lebih awal setelah kecocokan ke-5 ditemukan berkat mekanisme *early termination*.

6. Test Case 6

Input	
Sumber	URL
URL	https://github.com
Algoritma	BFS
Opsi Pencarian	Top N kemunculan, $N = 5$
CSS Selector	h1 + p
Output	
Hasil Visualisasi	

Tabel 12: Test Case 6

Analisis: Selektor `h1 + p` memilih elemen `<p>` yang tepat bersebelahan setelah `<h1>` pada level yang sama. BFS mengunjungi seluruh 1.942 node pada pohon berkedalaman maksimum 23 dalam waktu 2 ms, dan hanya menemukan 1 kecocokan. Hal ini menunjukkan bahwa struktur halaman GitHub sangat jarang memiliki pola `<h1>` diikuti langsung oleh `<p>` sebagai sibling.

7. Test Case 7

Input	
Sumber	Raw HTML
Konten HTML	<pre> <html> <body> <div class="container"> <section> <p class="text">Dalam section</p> </section> <p class="text">Langsung dalam div</p> </div> <p class="text">Di luar div</p> </body> </html> </pre>
Node A	<p class="text"> di dalam <section> (node-5)
Node B	<p class="text"> langsung di dalam .container (node-6)
Output	
Hasil Visualisasi	
LCA yang Dihasilkan	<div class="container">
Kedalaman Node A	4 (html → body → div → section → p)
Kedalaman Node B	3 (html → body → div → p)

Tabel 13: Test Case 7

Analisis: Algoritma *binary lifting* menyamakan kedalaman Node A ke level Node B terlebih dahulu, kemudian mengangkat keduanya bersama hingga bertemu di <div class="container"> sebagai LCA.

8. Test Case 8

Input	
Sumber	Raw HTML
Konten HTML	<pre><html> <body> <div id="root"> <section id="mid"> <article id="leaf">Konten</article> </section> </div> </body> </html></pre>
Node A	<div id="root"> (node-3)
Node B	<article id="leaf"> (node-5)
Output	
Hasil Visualisasi	
LCA yang Dihasilkan	<div id="root"> (Node A itu sendiri)
Kedalaman Node A	2 (html → body → div)
Kedalaman Node B	4 (html → body → div → section → article)

Tabel 14: Test Case 8

Analisis: Setelah penyamaan kedalaman, Node B yang diangkat akan bertemu tepat di posisi Node A sehingga kondisi $a == b$ langsung terpenuhi. LCA yang dikembalikan adalah Node A itu sendiri, membuktikan implementasi menangani kasus ini dengan benar tanpa galat.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Tugas besar ini berhasil mengimplementasikan aplikasi penelusuran pohon DOM berbasis algoritma BFS dan DFS untuk mencari elemen HTML berdasarkan CSS selector. Kedua algoritma diimplementasikan secara iteratif menggunakan struktur data antrian dan tumpukan eksplisit, sehingga aman digunakan pada pohon DOM berukuran besar tanpa risiko *stack overflow*. Hasil pengujian menunjukkan bahwa BFS efektif menemukan elemen yang berada pada level dangkal terlebih dahulu, sedangkan DFS lebih hemat memori dan cocok untuk penelusuran pohon yang dalam. Mekanisme pencocokan CSS selector mendukung empat jenis kombinator, *descendant*, *child*, *adjacent sibling*, dan *general sibling*, yang memungkinkan penelusuran dengan ekspresi selector kompleks. Selain itu, fitur bonus berupa animasi penelusuran, paralelisasi pencocokan menggunakan *goroutine*, dan algoritma LCA dengan teknik *binary lifting* berhasil diimplementasikan dan berjalan sesuai spesifikasi. Secara keseluruhan, aplikasi yang dibangun memenuhi seluruh spesifikasi wajib yang ditetapkan dan memberikan visualisasi penelusuran pohon DOM yang interaktif dan informatif.

5.2 Saran

- Pencocokan CSS selector saat ini hanya mendukung satu kelas per selektor. Dukungan terhadap *multi-class selector* (contoh: `.class1.class2`) dapat ditambahkan untuk meningkatkan kelengkapan fitur.
- Jumlah *worker goroutine* saat ini dikodekan secara tetap (`workerCount = 4`). Penyesuaian jumlah *worker* secara dinamis berdasarkan ukuran pohon DOM dapat meningkatkan efisiensi pada berbagai kondisi masukan.
- Fitur LCA saat ini bergantung pada pohon hasil traversal terakhir. Penambahan kemampuan memilih pohon dari riwayat traversal sebelumnya akan meningkatkan fleksibilitas penggunaan.
- Beberapa halaman web besar memerlukan waktu *scraping* yang cukup lama. Penambahan mekanisme *caching* hasil *scraping* dapat mengurangi latensi pada pengujian berulang terhadap URL yang sama.

Pernyataan Tidak Melakukan Kecurangan

Dengan ini, kami yang bertanda tangan di bawah ini menyatakan bahwa laporan Tugas Besar 2 IF2211 Strategi Algoritma yang berjudul **“Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model”** ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 25 April 2026

Samuelson Dharmawan T.

13524001

Edward David Rumahorbo

13524036

Reinhard Alfonzo Hutabarat

13524056

Daftar Pustaka

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, edisi ke-4. Cambridge, MA: MIT Press, 2022.
- [2] WHATWG, “DOM Living Standard,” [Online]. Tersedia: <https://dom.spec.whatwg.org/>. [Diakses: April 2026].
- [3] WHATWG, “HTML Living Standard,” [Online]. Tersedia: <https://html.spec.whatwg.org/>. [Diakses: April 2026].
- [4] E. Etemad dan T. Çelik (Ed.), “Selectors Level 4,” W3C Working Draft, W3C, November 2022. [Online]. Tersedia: <https://www.w3.org/TR/selectors-4/>. [Diakses: April 2026].
- [5] M. A. Bender dan M. Farach-Colton, “The LCA Problem Revisited,” dalam *Proc. LATIN 2000: Theoretical Informatics*, Lecture Notes in Computer Science, vol. 1776, Springer, Berlin, 2000, hal. 88–94. https://doi.org/10.1007/10719839_9.
- [6] R. Sedgewick dan K. Wayne, *Algorithms*, edisi ke-4. Upper Saddle River, NJ: Addison-Wesley Professional, 2011.
- [7] Asisten Laboratorium Ilmu dan Rekayasa Komputasi, *Spesifikasi Tugas Besar 2 IF2211 Strategi Algoritma: Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model*, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, v1.0, 2026.

Lampiran

1. Repositori GitHub:

https://github.com/samuelsondhrm/Tubes2_BOELANGOSONK-V2

2. Link *Deployment* Azure VM:

<http://boelangosonk.malaysiawest.cloudapp.azure.com/>

3. Tabel Pembagian Tugas:

Nama	NIM	Tanggung Jawab
Edward David Rumahorbo	13524036	<i>Backend</i> : HTML scraper (<i>net/http</i>), algoritma BFS, HTML parser (tokenisasi + konstruksi pohon DOM), endpoint <i>POST /api/scrape</i> .
Reinhard Alfonzo Hutabarat	13524056	<i>Backend & Frontend</i> : <i>CSS Selector engine</i> (<i>MatchesSelector</i>), algoritma DFS, <i>multi-threading</i> goroutine untuk pencocokan paralel, endpoint <i>POST /api/traverse</i> .
Samuelson Dharmawan Tanuraharja	13524001	<i>Backend & Frontend</i> : algoritma LCA <i>Binary Lifting</i> , <i>in-memory tree store</i> , <i>SSE streaming</i> (<i>traverse_stream.go</i>), <i>frontend</i> animasi traversal (<i>useTraversal</i> , <i>PlaybackControls</i>), panel LCA (<i>LCAPanel</i>), <i>Docker + Docker Compose</i> , <i>deployment</i> Azure VM + <i>Nginx</i> .

Tabel 15: Tabel pembagian tugas anggota kelompok BOELANGOSONK-V2

4. Tabel Checklist Pembuatan Fitur:

No	Poin	Ya	Tidak
1	Aplikasi berhasil dikompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	